

AI Elite Program — Customer Presentation Document

Autonomous IBM Power Porting Triage Agent with Multi-Pillar Architecture Auditing

Initiative: AI Elite Program Prototype Submission	Target Architecture: IBM Power (ppc64le)
Primary Personas: Technical Pre-Sales & Solution Architects	Runtime Platforms: RHEL 8/9, OpenShift (OCP), Ubuntu, SLES

1. Customer Profile & Target Personas

Customer Profile: Global enterprises running high-throughput transactional, time-series, and data/AI workloads (financial services, telecom, healthcare, and retail) evaluating platform modernization from commodity x86 infrastructure to **IBM Power (ppc64le)** on RHEL and OpenShift.

- **IBM Technical Pre-Sales Specialists & Solution Architects:** Frontline technical sellers who must rapidly qualify migration feasibility, size porting timelines, and build client confidence during competitive evaluations.
- **Enterprise Decision-Makers (CIO / CTO / VP Infrastructure):** Stakeholders requiring transparent risk qualification, timeline predictability, and platform ROI before approving modernization contracts.
- **IBM Power Porting Lab & Ecosystem Engineers:** Specialized engineering squads responsible for compiling, tuning, and verifying unported packages, who need structured, actionable technical backlogs.

2. Business Problem & Pre-Sales Friction

While IBM Power offers unmatched hardware throughput (high memory bandwidth, SMT8 core threading, and hardware Matrix Math Accelerators), **pre-sales qualification creates severe friction in the sales pipeline:**

- **Unstandardized & Heterogeneous Inputs:** Customers provide dependency specifications in fragmented formats: CycloneDX/SPDX SBOM JSONs, multi-stage Dockerfiles, Python requirements, Compose/Kubernetes YAMLS, or informal notes.
- **Fragmented Ecosystem Registries & Version Drift:** Determining package availability requires tedious manual searches across RHEL BaseOS/AppStream, EPEL 9, Quay.io, Docker Hub, PyPI, and community channels (Conda-forge, IBM Open-CE). Often, generic version queries give false confidence when the customer's *specific version tag* lacks a ppc64le binary.
- **The 'Iceberg' Estimation Gap:** When a C/C++ library or database engine lacks an official prebuilt binary for ppc64le, pre-sales teams cannot see beneath the surface. Uncovered build-time dependencies (e.g. jemalloc needing 64KB page size tuning) or unported x86 SIMD intrinsics (AVX2/AVX-512) surface late, resulting in costly project overruns.
- **Opaque Effort Sizing:** Traditional sizing relies on subjective guesswork, leading to 2x–5x variance and eroded trust between sales teams and client engineering leads.

3. Quantified Business Impact

Operational Dimension	Current Manual Process	AI Agent Prototype Impact
Qualification Turnaround Time	5 to 10 Business Days across manual catalog lookups and emails	< 30 Seconds automated end-to-end qualification
Porting Lab Bandwidth Waste	~40% of senior engineer time spent on routine initial triage	Zero wasted hours on routine triage; lab only receives scoped specs
Deal Velocity & Friction	Prolonged sales cycles; customer hesitation due to unknown porting risks	Immediate sales confidence with instant traffic-light readiness scoring
Effort Sizing Accuracy	Subjective guesswork prone to 2x–5x variance from hidden dependencies	Transparent 3-Pillar Sizing (Build + Engineering + Test PD)
Container & CI Visibility	Multi-stage Dockerfiles and CI workflows ignored until deployment	Automated multi-stage Dockerfile & CI matrix scanning
Executive Proposal Deliverables	Days spent assembling custom slide decks and proposal memos	1-Click Executive Memo & CSV Backlog generated autonomously

4. Proposed Opportunity: Autonomous AI Agent Architecture

- The **Autonomous IBM Power Porting Triage Agent** combines deterministic ecosystem probing with generative sales synthesis, delivering instant qualification, transparent effort sizing, and executive deliverables:
- **Universal Ingestion & Multi-Stage Dockerfile Parsing:** Automatically ingests CycloneDX/SPDX SBOMs, Python requirements, Compose/Kubernetes manifests, and raw multi-stage Dockerfiles (FROM ... AS builder), validating base images for every build and runtime stage.
 - **Multi-Source Prober with Version-Aware Intelligence:** Executes real-time live queries against Quay.io, Docker Hub v2 API, RHEL/EPEL 9 repos, and PyPI. Detects version mismatches (e.g. requested tag unavailable on ppc64le), providing actionable upgrade/downgrade advice with clickable verification URLs.
 - **Deep Transitive Build Dependency Analysis ('Solving the Iceberg'):** Recursively inspects build requirements (BuildRequires, submodules, header libraries) for unported packages to uncover hidden dependencies before compilation starts (e.g. scoping jemalloc-ppc64le for RocksDB).
 - **Hardware Architecture Sensitivity Auditing:** Audits source code and build configs for CPU architecture friction: flags x86 SIMD (AVX2/AVX-512) intrinsics (recommending SIMD to Power VSX translations), 64KB page size sensitivities (jemalloc tuning), and inline assembly (x86 pause mapped to Power or 27,27,27).
 - **Calibrated SIMD Volume Bracket Scaling:** Scales engineering effort based on vector instruction volume across 5 standardized brackets: 0–50 inst (1.0x), 51–200 inst (1.5x), 201–500 inst (2.5x), and >500 inst (4.0x multiplier), reflecting the code surface requiring inspection and porting.
 - **4-Tier SIMD-to-VSX Porting Complexity Classification:** Classifies vector adaptation into 4 rigorous engineering tiers: DIRECT (1.0x), SIMDE_COMPATIBLE (1.5x), PARTIAL_REWRITE (2.5x), and FULL_REDESIGN (4.0x for AVX-512 gather/scatter or Power MMA kernels).
 - **Dedicated Test Suite & Validation Effort Sizing:** Discovers test frameworks (GTest, pytest, Catch2, Testcontainers) and allocates dedicated test validation effort (test_effort_pd) to guarantee regression verification is sized alongside compilation.
 - **Git Repository Architecture Posture & CI Matrix Scanning:** Inspects upstream Git repositories for existing Power markers (#ifdef __powerpc__) and audits GitHub Actions / GitLab CI matrices for active ppc64le runners, applying an effort adjustment factor (0.7x for mature Power CI to 1.3x for x86-only codebases).
 - **Compose, Kubernetes & Infrastructure Image Verifier:** Parses Docker Compose files, Kubernetes manifests, and Helm charts to verify all ancillary service images (databases, sidecars, log forwarders) on ppc64le.
 - **Generative Executive Proposal Engine:** Uses Google Gemini (with resilient multi-model fallback across gemini-3.5-flash and gemini-3.5-flash-lite) to draft client-ready strategic value proposals highlighting Power memory bandwidth, SMT8 threading, and enterprise ROI.

Universal Manifest Ingestion SBOM, Dockerfile, Compose, Git URL, Free-text	Deterministic Prober & Scoper Quay, Docker Hub, RHEL 9, PyPI, Deep Build Tree	Architecture & CI Posture Audit SIMD Brackets, 4-Tier VSX, 64K Pages, CI Matrix	Executive Synthesis & Sizing Build+Eng+Test PD, Traffic Light, 1-Click Memo
--	---	---	---

5. Transparent Effort Sizing Methodology & Engineering Derivation

To replace subjective estimation with mathematical rigor, the agent computes porting effort using a standardized **3-Pillar Sizing Formula** that isolates compilation, hardware adaptation, and test validation:

Formula: Total Person-Days (PD) = Build Effort + Engineering Adaptation Effort + Test Effort
Where: Engineering Effort = Base Engineering × SIMD Count Multiplier × Complexity Multiplier × Repo CI Factor

Sizing Parameters & Multiplier Matrix

Sizing Dimension	Bracket / Classification Tier	Multiplier	Engineering Rationale & Scope
SIMD Instruction Volume	0 – 50 instructions	1.0x	Reflects the total vector instruction surface requiring manual inspection, inline substitution, and micro-benchmarking.
	51 – 200 instructions	1.5x	
	201 – 500 instructions	2.5x	
	> 500 instructions	4.0x	
Porting Complexity Tier	DIRECT	1.0x	From 1:1 intrinsic substitution, to SIMD header mapping, to 128-bit vs 256-bit loop refactoring, up to full AVX-512 redesign to Power MMA.
	SIMDE_COMPATIBLE	1.5x	
	PARTIAL_REWRITE	2.5x	
	FULL_REDESIGN	4.0x	
Repository CI Posture	Power CI present (ppc64le runner)	0.7x (Discount)	Accounts for upstream willingness and infrastructure to validate, accept, and maintain Power patches.
	Neutral / Mixed markers	1.0x (Baseline)	
	Hardcoded x86 / No Power CI	1.3x (Friction)	
Test Suite Validation	Standard Unit Tests	0.5 – 1.0d	Ensures regression verification, test container availability, and performance validation are fully accounted for.
	Integration / Benchmarks	1.5 – 2.0d	
	Missing Native Test Harness	+2.0d	

Worked Enterprise Sizing Derivations

Target Component	Component Analysis & Findings	Step-by-Step Derivation Formula	Total PD
simdjson 3.6 (C++ JSON Parser)	- Build: CMake toolchain (1.0d) - SIMD: 320 AVX2 instructions - Tier: SIMDE_COMPATIBLE - Test: cxxopts + benchmark (1.0d)	Build: 1.0d Engineering: 2.0d base × 2.5 (vol) × 1.5 (tier) × 1.0 (CI) = 7.5d Test: 1.0d Sum: 1.0 + 7.5 + 1.0 = 9.5 ≈ 10 PD	10 PD
RocksDB 8.6 (Storage Engine)	- Build: CMake (1.5d) - Transitive: jemalloc-64k (1.5d) - Arch: Inline asm yield (1.5d) - Test: GTest harness (1.0d)	Build: 1.5d Transitive Build Dep: 1.5d (jemalloc 64k config) Engineering: 1.5d base × 1.0 × 1.0 = 1.5d Test: 1.0d Sum: 1.5 + 1.5 + 1.5 + 1.0 = 5.5 ≈ 6 PD	6 PD

6. Demonstrated Workload Scenarios & Validation

The prototype was tested and validated across four production-grade enterprise migration workloads:

Scenario / Workload	Stack Components	Agent Triage Findings & Recommendations	Score & Sizing
Enterprise Cloud Microservices	Nginx, Redis, PostgreSQL, Strimzi Kafka, Node.js	All components have verified native ppc64le container images in Docker Hub and Quay.io. Multi-stage Dockerfile audit confirmed ppc64le base images.	100% GO 5 Person-Days (Smoke & staging)
AI / ML Vector Pipeline	PyTorch, NumPy, Pandas, Scipy, Custom C++ DSP	PyTorch routed to IBM Open-CE with Power MMA acceleration; custom DSP kernel scoped with SIMDe AVX2->VSX remediation. Test suite validated.	95% GO 6 – 8 Person-Days (1d Build + 4d Eng + 2d Test)
High-Throughput Storage Engine	RocksDB 8.6, simdjson 3.6, Redis, zlib	Deep dependency iceberg scoped: 6 build deps, including 64KB-page tuned jemalloc and SIMD vector translation. Full transparent derivation provided.	30% CAUTION 12 – 16 Person-Days (Multi-package iceberg)
Legacy Financial Analytics	Intel MKL, Python 3.9, Proprietary Risk Calc	Proprietary closed-source x86 blocker flagged; automated recommendation to substitute with IBM ESSL or OpenBLAS.	HIGH RISK Arch Substitution required

7. Strategic Value to IBM & Program Alignment

This prototype directly addresses the core mission of the **AI Elite Program** by applying advanced agentic AI to eliminate real-world enterprise sales friction:

- Accelerates Infrastructure Revenue:** Eliminates the 5-to-10 day qualification bottleneck, arming client teams with authoritative migration roadmaps within seconds of first contact.
- Optimizes Engineering Allocation:** Prevents the IBM Power Porting Lab from wasting bandwidth on off-the-shelf components, focusing scarce specialist time exclusively on scoped unported libraries.
- Demonstrates Hybrid AI Leadership:** Seamlessly combines deterministic knowledge-graph probing, container registry APIs, and multi-model LLM reasoning to solve mission-critical systems challenges.
- Actionable Developer Deliverables:** Generates one-click executive 1-page proposals alongside structured CSV engineering backlogs ready for immediate JIRA ingestion.

Submission Note: Prototype is fully implemented and operational, containerized via Podman / Docker Compose, validated with a 100% passing test suite across all 7 gap features, and available for live demonstration.